

ПОЗИЦИЯ

13 ИЮЛЯ 2025 ГОДА

Горький урок оценки больших языковых моделей

Превращение экспертного суждения в непреодолимый барьер. Потому что при оценке больших языковых моделей масштабирование заботы важнее масштабирования вычислений.



Чарльз О'Нил (Базеттен)



Мудит Джаясекара (Басетен)



Макс Киркби (Baseten)

В истории искусственного интеллекта есть известный принцип, получивший название «горький урок». Его сформулировал Рич Саттон. Согласно этому принципу, наибольший прирост производительности достигается не за счет сложных знаний, разработанных людьми, а за счет общих методов, использующих огромные вычислительные мощности. Урок прост и поучителен: в долгосрочной перспективе побеждает метод проб и ошибок.

Когда дело доходит до оценки сложных систем искусственного интеллекта, которые мы создаем сегодня, соблазнительно применить тот же подход. Хочется верить, что мы можем автоматизировать процесс понимания, увеличив вычислительные мощности: что более крупная модель, такая как GPT-4, с продуманной подсказкой, может служить объективным и масштабируемым судьей.

Это опасная иллюзия.

При оценке LLM появляется новый и перевернутый горький урок: наиболее эффективный путь к надежному сигналу лежит не через масштабирование вычислений, а через *тщательность* масштабирования. Прогресс достигается благодаря тщательным, структурированным и глубоко человеческим усилиям по определению того, что именно мы пытаемся измерить. Успех в прикладном ИИ зависит от того, насколько быстро вы можете выполнять итерации, а скорость итерации определяется единственным, неумолимым узким местом: вашей способностью определить, улучшило ли изменение ситуацию или ухудшило ее.

Погоня за вычислительными упрощениями без такой основы — это путь в никуда, погоня за призрачными улучшениями, основанными на шаткой и ненадежной системе координат.

Оценка — это самое важное действие при разработке искусственного интеллекта. И горький урок, который можно извлечь из этого процесса, заключается в том, что нельзя автоматизировать процесс, не научившись ясно мыслить.

Мираж оценки больших языковых моделей

На первый взгляд, решение для масштабирования оценки — это «LLM в роли судьи». Идея заманчива: спросите у мощной модели, такой как GPT-4, насколько хорош результат работы вашей системы. Это кажется объективным и бесконечно масштабируемым решением. На самом деле этот подход зыбок. Готовые решения на основе больших языковых моделей — ненадежная основа для разработки, страдающая от противоречий, обусловленных фундаментальными принципами работы этих моделей.

Нестабильность авторегрессионной генерации

Большие языковые модели генерируют текст по одному токену за раз, при этом каждый новый токен зависит от предыдущих. Этот авторегрессионный процесс по своей сути является стохастическим. Многие считают, что установка `temperature=0` решает эту проблему, заставляя модель всегда выбирать наиболее вероятный следующий токен. Это позволяет получать детерминированные и воспроизводимые результаты.

Но это заблуждение. Большинство моделей не генерируют текст оптимальным образом на `temperature=0`, а незначительные факторы в стеке глубокого обучения (как мы недавно убедились на примере [скандала](#) между vLLM и HuggingFace) могут изменить арифметику с плавающей запятой настолько, что чаша весов склонится в пользу одного токена перед другим. Один-единственный другой токен на начальном этапе может привести к совершенно иному результату.

Что еще важнее, детерминизм не решает основную проблему — чрезмерную чувствительность. Окончательный вердикт, вынесенный большой языковой моделью, может зависеть от смехотворно незначительных, семантически нерелевантных изменений в запросе на оценку (см. [здесь](#), [здесь](#) и [здесь](#)).

- **Предвзятость в оценке:** простой перестановкой критериев в вашей оценочной шкале можно изменить результат. Большая языковая модель может придавать большее значение первому критерию, который она считает, то есть оценка по принципу «краткость, а затем точность» может дать другой результат, чем оценка по принципу «точность, а затем краткость».
- **Чувствительность к формулировкам:** сравните эти две инструкции: «Ответ должен соответствовать исходному тексту» и «Ответ не должен содержать никакой информации, которой нет в исходном тексте». Для человека они практически идентичны. Для больших языковых моделей это разные последовательности токенов, которые могут запускать совершенно разные логические цепочки и, следовательно, приводить к разным суждениям.

Постоянная «подгонка» под стабильную конфигурацию — это изнурительно. В итоге вы просто ищете инструмент для измерений, который не будет шататься в руках.

Верность и готовность помочь

Помимо нестабильности, нейросети-судьи часто не улавливают тонкие, не связанные друг с другом аспекты того, что делает ответ хорошим. Два самых важных аспекта — это точность и полезность.

- **Достоверность** показывает, опирается ли ответ *только* на предоставленные исходные документы. Ответ не должен содержать выдуманных фактов (галлюцинаций) или дополнительной информации.
- **Полезность** показывает, является ли ответ актуальным, исчерпывающим и полезным для пользователя.

A response can be perfectly faithful yet utterly useless. Ask a system, "What are the penalties for a missed payment according to this lease agreement?" and it might respond, "Clause 4.2 of the lease agreement addresses missed payments." This is 100% faithful to the text but provides zero help. A generic LLM judge, tasked with checking for factual accuracy, might score this a perfect 10/10.

This leads to failure modes that simple evaluations miss:

- **False Negatives:** The system incorrectly states that the source document doesn't contain the answer. This is often caused by poor retrieval or the model's limited attention over large contexts, or the "lost in the middle" problem where information in the middle of a long document is ignored.
- **False Positives:** The system hallucinates an answer that sounds plausible but doesn't exist in the source document.

A good evaluation system must be able to distinguish between these failure modes across thousands of examples, calibrated to your specific task. A generic LLM judge simply cannot.

The hardest thing to admit: you can't use generic evals

I think the majority of companies we've spoken to and worked with have assumed that at least part of their eval stack can reuse generic or at least non-customised eval prompts when testing for things like hallucinations and completeness in their model outputs. Unfortunately, the bitter lesson of evals is that this just doesn't work. For a lot of the reasons above, at some point you just have to bite the bullet and LOOK AT YOUR DATA (as [Hamel Husain would say](#)) and determine what you actually want to evaluate. Only then can you actually start sculpting the contours of the evaluation which reflects what a domain expert in your company would consider to be the ground-truth.

The Baseten flywheel: forging a compounding moat

To escape this cycle, you need to treat evaluation as the core product. You need to build a system (we call it a harness) that is more than just a metric. It must become the ground truth that your entire development process orbits around. This is what we build at Baseten.

Ground truth doesn't exist in a dataset

A universal evaluation metric doesn't exist. Ground truth is unique to your task, your domain, and your users. That's why we begin with a forward-deployed model. Our top engineers embed directly with your team for 1-4 weeks. They don't just ask for a dataset; they sit side-by-side with your domain experts to understand their workflow, their heuristics, and their definition of "quality."

We use a process of critique shadowing. Your principal domain expert makes simple, binary Pass/Fail decisions on representative traces, but provides detailed critiques explaining their reasoning. This forces clear thinking, is immediately actionable, and is far faster than debating arbitrary 1-5 scales. This expert judgment becomes the seed of your evaluation harness, along with any user feedback you've collected.

Stop with your monolithic eval prompts

A single evaluation question like "Is this summary good?" is impossible to answer consistently. To solve the sensitivity and random neglect problems, we break the task down. Instead of one large, complex prompt sent to a single powerful model, we build a **rubric fan-out** (credit to Haize Labs for [coining this term](#)).

Imagine evaluating a RAG system's answer. We deconstruct the task into a series of metrics you want to monitor, and then each of those metrics into a set of independent checks:

- **Individual Checks:** A small, fast, possibly fine-tuned model does *only one thing*. One model checks if claim #1 is supported by the context. Another checks if claim #2 is. A third checks for a polite tone. A fourth checks for conciseness. Each model has a narrow, unambiguous task, making it robust to prompt variations.
- **Metrics:** The binary outputs of these checks are then aggregated into the higher-level metrics they were derived from.
- **The Harness Score:** Finally, these metrics are weighted and combined into a single, observable score that gives you an immediate, top-level signal of system health.

This granular structure allows us to have absolute certainty that every single criterion in your rubric is evaluated, every single time. It transforms an unreliable process into a deterministic, mechanistic one.

Tame the variance

To solve the inherent stochasticity of LLMs, we treat evaluation as a statistical process. We deploy **ensembles of judges** for each granular check, using techniques like mean-pooling or self-consistency where multiple attempts are made and the most common outcome is chosen as the final verdict. As a wide body of research has shown, this approach drastically smooths out variance.

It also allows us to use an ensemble of weaker, faster, and cheaper models to achieve results that are more stable and aligned with human judgment than a single, expensive call to a frontier model. The result is an evaluation harness that is not only more reliable, but also orders of magnitude more efficient to run at scale.

The compounding loop

This is the bitter lesson of LLM evals in practice. The compounding moat of quality isn't dug with more compute, but with more rigour. It's built by embracing the hard work of thinking from first principles about what "good" actually means for your specific task, and then mechanising that deep understanding into a system you can trust. (We even use this evaluation harness to drive continuous improvement of a custom model for your task, but that's a story for another day.)

This flywheel is the direct embodiment of this lesson. It is a system for scaling *care*, for turning meticulous human judgment into a reliable, high-velocity engine for improvement. It builds a defensible moat of quality and performance that you can use to improve the

models doing your task and compounds over time, making it nearly impossible for competitors to catch up.

The next generation of breakout AI companies won't be won by those who are quickest to adopt the latest model. It will be won by those who build the most superior engine for improvement.

It's a harder path, but it's the only one that leads to a durable advantage.

Other research



POSITION

The Michael Scott Paper Company of AI

Open-source doesn't need to wait out the frontier labs. It just has to wait out the gap.

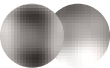
 CHARLES O'NEILL



POSITION

Continual learning and the post monolith AI era

The cost of continual learning scales with generality.



CHARLES O'NEILL • 1 other

POSITION

If we can't design neat latent structures, then maybe we can Bitter Lesson it through self-study

They very existence of Claude's Constitution shows that self-study, to a certain extent, just works.



CHARLES O'NEILL

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Продукт

Специализированное логическое устройство

API-интерфейсы моделей

Обучение

Frontier Gateway

Платформа логического вывода Baseten

Исполняемые среды моделей

Инфраструктура

Мультиоблако

Варианты развертывания

Облачное

Собственное

Гибридное

Встраиваемая инженерия

Инженеры , развернутые на переднем крае

Modalities

Transcription

Image Generation

Text-to-speech

Large language models

Compound AI

Embeddings

Industries

Enterprise

Healthcare

Разработчик

Библиотека моделей

Документация

Список изменений

Ресурсы

Исследования

Клиенты

Блог

Руководства

Мероприятия

Партнеры

Калькулятор сбережений

Центр доверия

О нас

Стартапы

Вакансии

Контакты

Популярные модели

GLM 5.2

Kimi K2.7 Code

DeepSeek V4

GPT OSS 120B

Whisper Large V3

NVIDIA Nemotron 3 Ultra

Explore all

Legal

Terms and Conditions

Privacy Policy

Service Level Agreement

● ALL SYSTEMS NORMAL

© 2026 BASETEN

